A Calculus of Heterogeneous Software Assurance: Canonical Evidence, Independent Derivations, and Exact Correspondence

bordumb

3 September 2026

Abstract

The output of a software-assurance pipeline is ordinarily a vector of tool exit codes subsequently collapsed to a Boolean. Such a quotient is unsound as an account of justification: finite testing, bounded exploration, theorem checking, source correspondence, artifact reproduction, and human review have different quantifiers, authorities, and subjects. We present the semantic core of Proofbound as a typed calculus in which claims, evidence, facts, assumptions, artifacts, derivations, and publication policies inhabit distinct syntactic categories. Its intermediate language is backend-neutral and canonically encoded; its evidence language is a closed sum; its derivations are independently replayable directed acyclic graphs; and its artifact, provenance, cache, and policy relations are exact joins rather than informal annotations. Publication is therefore a derived judgment $\Gamma; E; D; \Pi \vdash c \Downarrow \delta$, never an adapter assertion.

We evaluate this design by preregistered, falsifier-driven experiments over checked-in positive, holdout, and adversarial corpora. In EXP-0005, independent Rust and Python implementations agreed on 20 positive projections, 20 exact adversarial rejections, and 15 canonical hash vectors; a later projection represented 45 portable evidence records without silently promoting two legacy sampling records. EXP-0006–0008 establish, then falsify and repair, a cross-framework sampling abstraction: tool identity and seed are insufficient; a sound record also requires backend plans, authority-indexed observations, availability, and explicit consumers. In EXP-0009, the implementations agreed on all 500 valid and 500 single-mutation adversarial derivation programs over six routes, eleven rules, and sixteen attack classes, with no backend-named branch in the common rules. These are finite validation results, not proofs of soundness or completeness. Five of sixteen EXP-0005 losslessness obligations remain partial. The resulting contribution is a precise semantic kernel and a conservative adoption argument: Proofbound should first compose Python, TypeScript, Rust, Lean, Kani, Aeneas, and related systems; native syntax is warranted only if it preserves this calculus while eliminating demonstrable accidental complexity.

1. Introduction

Let T be a set of assurance tools and let the conventional CI abstraction be $q : T \to \{0, 1\}$. A release gate commonly computes

$$Q(t_1, \dots, t_n) = \bigwedge_{i=1}^{n} q(t_i).$$

The map q forgets precisely the information on which assurance depends: the quantifier discharged, the subject examined, the execution controls, the authority of an observation, the residual assumptions, and the relation between a formal model and released bytes. The defect is not that Q is Boolean; a publication decision must eventually be discrete. The defect is that Q is computed before an explicit semantics of justification has been recovered.

A unit test establishes an observation at one registered execution. A sampled property records a finite campaign under a generator, predicate, seed, budget, and backend plan. A bounded model checker establishes a proposition only within declared bounds and assumptions. A proof assistant may check a universal theorem in a formal environment. A reproducible build establishes equality of artifact bytes across declared executions. These conclusions are neither synonymous nor freely coercible.

In contemporary development these facts are distributed among workflow files, tool-specific reports, source annotations, artifact stores, and dashboards. Composition is typically performed through booleans, job dependencies, and

prose. That representation is too weak for assurance: it can hide quantifier changes, confuse a model theorem with a property of shipping bytes, omit an execution-relevant parameter, or report missing diagnostic telemetry as if a claim had weakened. Specification 0001 therefore treats formal standing, subject linkage, and assumption burden as separate facets and makes status a derived output rather than an adapter-authored label (Proofbound project 2026g).

The Proofbound programme asks whether this information admits a compact, language-independent calculus and, subsequently, a satisfactory surface language (Proofbound project 2026f). The order is deliberate. Existing Python, TypeScript, Rust, Lean, Kani, Aeneas, and release records supply the semantic observations; the calculus must explain them before a frontend is permitted to conceal them.

This paper contributes: (i) a many-sorted assurance calculus separating evidence families, authority, availability, correspondence, and policy; (ii) a canonical Assurance IR and replay semantics for backend-independent derivation traces; (iii) finite differential validation against Experiments 0005–0009, including a holdout that falsifies an initially plausible sampling contract; and (iv) a conservative language-realization criterion that confines backend diversity to typed boundaries.

The terminology is intentionally strict throughout. *Proof* refers only to an appropriately checked theorem. Tests, static analyses, randomized sampling, mutation witnesses, finite enumerations, and bounded model checks are evidence, but they are not described as proofs. A theorem about a model is not described as a shipping claim without a registered correspondence to the relevant source or artifact.

2. Motivation

2.1 Fragmentation is a semantic problem

The conventional pipeline is organized around tools. CI invokes a test runner, type checker, model checker, prover, packager, scanner, and policy step; each reports success in its own schema. A green workflow answers whether the configured commands returned an accepted process outcome. It does not necessarily preserve the claim, inventory, sampling regime, formal proposition, correspondence, assumptions, or exact published artifact associated with that outcome.

This is not merely a user-interface defect. Three kinds of information loss are structural.

1. **Strength loss.** Unlike evidence families collapse into `passed`. A sampled observation can then be retold as a universal guarantee; a static consistency check as functional correctness; or a bounded result without its bounds.
2. **Identity loss.** Tool names, filenames, or raw digests substitute for typed identities. The relevant relation may require a logical artifact name, digest, byte size, proposition identity, inventory, subject closure, and policy ID at once.
3. **Authority loss.** A value may be registered, observed, reviewed, derived, or unavailable. Flattening those origins permits configuration to masquerade as observation, a derived counter to masquerade as a measured one, or absence to masquerade as zero.

The repository supplies concrete instances. In EXP-0006, Vitest emitted strict JSON saying that an outer test node passed, yet that JSON omitted the nested fast-check seed, run count, generator identity, skip count, shrink count, and effective configuration. Hypothesis printed useful counts as terminal prose but did not emit the closed machine report required by the preregistration. Neither route could establish the proposed sampling contract merely from ordinary runner output (Proofbound project 2026b). Likewise, EXP-0005 found that locating an executable and reading its version did not establish readiness: sealed npm execution also needed dependency-fetch authority, while Lean evidence needed compiled project modules (`docs/experiments/0005-assurance-ir-extraction/captures/q1-completion-r1/index.json`).

2.2 Notification fatigue follows from tool-oriented state

Tool-oriented reporting makes every missing field or failed job look locally important. Assurance decisions are claim-oriented: an unavailable fact matters when a registered derivation consumes it. The distinction is precise. If an empirical-admission rule requires a completed-case count, making that count unavailable blocks the conclusion. If the same rule does not consume shrink telemetry, the absence of that telemetry should remain recorded but should not generate an admission alert. EXP-0008 and EXP-0009 exercise exactly this pair of cases; both independent implementations distinguished them (Proofbound project 2026d, 2026e).

The programme therefore treats reduced notification volume as a future empirical hypothesis, not an implemented benefit. Current prototypes establish only that the data model and rules can represent consequence-indexed absence in finite corpora. Whether this improves human decisions without hiding important signals is reserved for structured user study (Hypothesis H6 and workstream WS-UQ in `docs/research/proofbound-language/`).

2.3 The target: an account of justification

The proposed language is best understood as a language for *accounts of justification*. For claim c, a publication decision should be reproducible from the exact registered programme P, evidence facts E, derivation trace D, and policy π:

$$\text{publish}(c) = \text{check}_K(P, E, D, \pi),$$

where K is a small independent checker. The producer may discover evidence, run expensive tools, or search for proofs. It may not author the final status. The result is not a scalar confidence score. It is a structured judgment with separate formal, linkage, assumption, and policy information.

3. Related Work

3.1 Proof assistants and proof-carrying computation

Lean combines a programming language with an interactive theorem prover and a small kernel that checks elaborated proof terms (Moura and Ullrich 2021). Proof-carrying code and certifying algorithms similarly separate a potentially complex producer from a simpler consumer that checks a witness (Necula 1997; McConnell et al. 2011). Proofbound adopts this producer–consumer discipline for assurance derivations. It does not replace the prover: the Assurance IR must record the theorem, assumptions, evaluation mode, and correspondence needed to use prover output in a release decision.

The difference in level is central. A proof assistant answers whether a theorem follows in a formal environment. The assurance language must additionally answer which registered software claim cites that theorem, what subject it concerns, whether production source or bytes correspond to the model, which policy admits it, and which residual assumptions remain. A model theorem stays `MODEL_ONLY` without the distinct correspondence evidence required by the specification.

3.2 Verus, refinement systems, and translation

Verus integrates executable Rust, specifications, ghost state, and SMT-backed verification (Lattuada et al. 2024). Refinement-type systems such as Liquid Types enrich program types with logical predicates and automate important proof obligations (Rondon, Kawaguchi, and Jhala 2008). Aeneas instead translates a Rust subset into functional code for interactive theorem provers (Ho and Protzenko 2022). These systems can establish program properties more directly than an assurance orchestrator.

Proofbound's proposed role is orthogonal and compositional. Verus, Lean, Aeneas, or a future verified compiler can be a proof-producing backend, but their outputs enter through a typed boundary. The Assurance IR preserves proposition identity, tool and trusted-base identity, assumptions, translated inventory, and the correspondence actually established. It also composes that formal evidence with empirical evidence, artifact reproduction, and publication policy. The native language proposal is subject to a stop condition: if a small Verus, Lean, or Dafny component integrated through Proofbound is equally strong and simpler, native expansion should stop (`workstreams/native-runtime.md`).

3.3 Build systems and CI configuration

Build systems formalize tasks, dependencies, scheduling, and rebuild decisions; their design space includes static and dynamic dependencies and several kinds of rebuild strategy (Mokhov, Mitchell, and Jones 2018). CI configuration composes such tasks with remote execution and reporting. Proofbound also needs dependency and cache semantics, but its equivalence relation is stricter: reuse is valid only when the typed semantic and execution dependencies that justify an evidence fact remain equivalent. A cached process success is not interchangeable with evidence about a different claim, inventory, artifact, tool, assumption, or policy.

Nor is Proofbound intended to schedule every build. The proposed kernel checks meaning and derivation after adapters normalize observations. Existing build and CI systems remain execution substrates. Their logs and attestations become inputs, not the semantic definition of `TESTED`, `PROVED`, `ARTIFACT_BOUND`, or publication.

3.4 Policy languages and assurance cases

Authorization languages such as Cedar decide whether principals may perform actions on resources and are designed for analyzable policy evaluation (Cutler et al. 2024). Assurance cases, including Goal Structuring Notation, organize claims, arguments, and evidence (Kelly and Weaver 2004), while Assurance 2.0 emphasizes explicit defeaters and disciplined argument (Bloomfield and Rushby 2021). These are close intellectual relatives, but the intended judgment differs. Proofbound policy does not decide an application request and cannot change evidence strength; it selects which already-derived judgments permit publication. The IR also binds canonical bytes, source closures, cache inputs, and executable provenance, then requires a second implementation to replay the derivation.

Supply-chain systems such as in-toto bind steps and artifacts to actors and layouts (Torres-Arias et al. 2019). Such provenance is necessary but does not by itself state what behavioral proposition an artifact satisfies. Conversely, a source theorem without artifact correspondence supplies behavioral meaning but not a shipping-byte claim. Assurance IR attempts to represent the join rather than substituting either side for the other.

4. Research Questions

The programme defines stable hypotheses H1–H8 in `docs/research/proofbound-language/hypotheses.md`. The present paper isolates six questions about semantic adequacy, portable execution contracts, and independent checkability. A positive finite corpus does not universally quantify a result; a valid counterexample does refute the corresponding universal abstraction.

ID	Question	Decision procedure and evidence boundary
RQ1	Can existing routes compile into a smaller backend-neutral IR without semantic loss?	Bidirectional field audit in EXP-0005; 11/16 obligations complete
RQ2	Can unlike evidence remain a closed algebra with forbidden strength coercions?	Bounded passes in EXP-0005, EXP-0008, and EXP-0009
RQ3	What must a portable sampled-property contract retain?	EXP-0006 positive result, EXP-0007 falsifying holdout, EXP-0008 layered replacement
RQ4	Can complete derivations be backend-independent and independently checked?	EXP-0009, six routes and 500/500 generated corpus
RQ5	Can unavailable facts be reported according to their consequences?	Finite attacks in EXP-0008 and EXP-0009; no human study yet
RQ6	Can migration, cache, artifact, provenance, policy, and invalidation joins fail closed?	Partial evidence in EXP-0005; dedicated invalidation experiment not yet run

The asymmetry is methodological. Acceptance of n corpus members establishes $\forall x \in C_n.P(x)$ only for the finite registered set C_n. By contrast, a single $x^\star$ satisfying the preregistered domain predicate and $\neg P(x^\star)$ refutes $\forall x.P(x)$. EXP-0007 supplies precisely such a counterexample to the flat two-framework sampling contract.

5. System Model

5.1 One kernel, several producers and surfaces

The candidate architecture separates authoring, evidence production, normalization, derivation, and publication. Figure 1 shows the intended trust direction. Tool-specific adapters may be numerous; constructors and derivation rules in the common kernel must remain few and versioned.

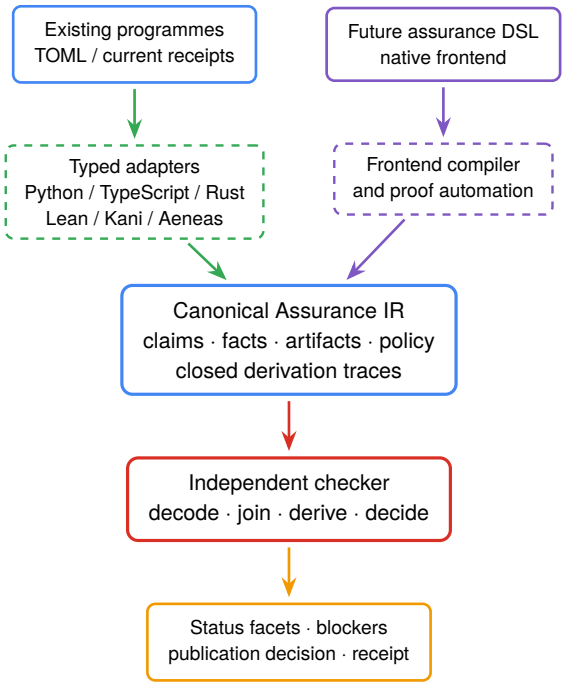

Figure 1: Candidate architecture. Backend-specific knowledge is permitted at typed conversion boundaries, not in common status derivation. The DSL and native frontend are proposals, not implemented production surfaces.

Formally, backend b is confined to a partial elaboration $\alpha_b : L_b \rightharpoonup \mathsf{AIR}$; all accepted terms then share the same canonicalizer, kernel, and policy projection:

$$L_b \xrightarrow{\alpha_b} \mathsf{AIR} \xrightarrow{\text{canon}} \mathsf{Bytes} \xrightarrow{K_\Sigma} \mathsf{Judgment} \xrightarrow{\Pi} \{\mathsf{Admit}, \mathsf{Block}\}. \tag{Arch}$$

The architectural invariant is that K_Σ has no case analysis on b. Backend-specific distinctions survive in typed plan or provenance values and may invalidate a record, but cannot introduce a new conclusion rule implicitly.

The checked-in production verifier already demonstrates a narrower version of the independence requirement. `crates/proofbound-verify/Cargo.toml` explicitly has no dependency on another `proofbound-*` crate; it decodes a closed receipt format and recomputes status without executing evidence backends. The research prototype is separate: `crates/proofbound-ir-prototype/` implements producer-side projection and candidate algebra checks, while `python/proofbound/assurance_ir_checker.py` is independently written and does not import the Rust types (Proofbound project 2026a, 2026e).

5.2 Claims, facets, assumptions, and policy

Let $\mathcal{C}, \mathcal{S}, \mathcal{E}, \mathcal{F}, \mathcal{A}, \mathcal{P}$, and $\mathcal{D}$ denote, respectively, claims, subjects, evidence terms, facts, artifacts, policies, and derivation traces. A claim is the record

$$c = \langle \iota, s, \varphi, X, \Omega, \pi \rangle \in \mathcal{C},$$

where ι is a stable identity, $s \in \mathcal{S}$ is a typed subject with a dependency closure, φ is machine-readable meaning, $X \subseteq \mathrm{Id}(\mathcal{E})$ cites evidence, Ω is the finite set of assumptions and exclusions, and $\pi \in \mathcal{P}$ is the selected publication policy. Reader-facing prose is deliberately not identified with φ.

The principal judgment has the form

$$\Gamma; E; D; \Pi \vdash c \Downarrow \langle F, L, A, B, \delta \rangle, \tag{1}$$

where Γ is the registered programme, E the available evidence store, D a closed trace, and Π the policy environment. The result contains formal standing F, linkage L, assumption burden A, blockers B, and publication decision $\delta \in \{\mathsf{Admit}, \mathsf{Block}\}$. Production Proofbound distinguishes (Proofbound project 2026g)

$$\begin{aligned} &F \in \{\mathsf{Open}, \mathsf{Tested}, \mathsf{BoundedChecked}, \mathsf{Proved}, \mathsf{Invalid}\}, \\ &L \in \{\mathsf{ModelOnly}, \mathsf{Transcribed}, \mathsf{Refined}, \mathsf{ArtifactBound}\}, \\ &A \in \{\mathsf{None}\} \cup \{\mathsf{Assumed}(\Omega) : \Omega \neq \varnothing\}. \end{aligned} \tag{2}$$

These facets form neither a confidence score nor an unrestricted total order. Instead, each evidence constructor has an *admissible conclusion set*. Policy may select from derived judgments but may not enlarge that set. If $\mathrm{ceil}(e)$ denotes the formal ceiling assigned to e, then every sound policy rule must satisfy the non-strengthening obligation

$$\frac{\Gamma; E; D \vdash c : F \qquad \Pi \vdash (c, F, L, A, B) \Rightarrow \delta}{F \preceq \mathrm{ceil}(E|_c)}. \tag{POL-CEIL}$$

Thus policy cannot reinterpret Tested as Proved, erase an assumption, or manufacture ArtifactBound from ModelOnly. EXP-0009 validates a smaller research slice and represents its bounded-check route with the slice's `tested` vocabulary; it is not a claim of complete parity with (2).

5.3 Closed evidence algebra

The draft Assurance IR (`docs/research/proofbound-language/assurance-ir-v1.md`) uses a discriminated sum rather than a common success record with optional fields. Abstracting from wire-level names, its evidence grammar is

$$\begin{aligned} e ::= {} & \mathsf{Example}(\bar{x}, o) \mid \mathsf{Sampled}(I, P, O) \mid \mathsf{Exhaustive}(U, O) \\ & \mid \mathsf{BMC}(\varphi, \beta, O) \mid \mathsf{Static}(q, O) \mid \mathsf{Mutation}(m, w) \\ & \mid \mathsf{Theorem}(\varphi, \Theta, p) \mid \mathsf{SourceCorr}(s_1, s_2, r) \\ & \mid \mathsf{ArtifactCorr}(\varphi, a, r) \mid \mathsf{Reproducible}(a, \bar{b}) \\ & \mid \mathsf{Transcription}(s_1, s_2, \Theta) \mid \mathsf{Review}(u, \sigma, v). \end{aligned} \tag{3}$$

Here I is sampling intent, P a backend plan, O an observation map, U a declared finite universe, β explicit exploration bounds, Θ a trusted base or assumption set, p a checked proof object or theorem reference, a an exact artifact, and r a registered correspondence. The grammar is *closed*: unknown alternatives do not decode, and no opaque extension may participate in status derivation. Table 2 states the associated quantifier discipline. Names remain research constructors rather than a frozen wire contract.

Constructor	What it can establish	What it cannot establish alone
`Example`	Exact registered examples passed	General or universal behavior
`SampledProperty`	A finite sampling campaign under an explicit intent and plan	Exhaustive or universal validity
`ExhaustiveFinite`	All members of a declared finite domain were checked	Behavior outside that domain
`BoundedModelCheck`	A proposition under explicit model-checking bounds and assumptions	An unbounded theorem
`StaticConsistency`	A registered analyzer accepted its exact inventory/configuration	Functional correctness
`MutationWitness`	A registered test distinguishes one registered mutant	General mutation adequacy or correctness
`UniversalSourceProof`	A theorem about the identified formal source subject	Shipping-byte correspondence
`SourceCorrespondence`	A registered relation between source representations	Artifact correspondence unless separately composed

Constructor	What it can establish	What it cannot establish alone
`ArtifactCorrespondence`	A theorem/claim is joined to an exact artifact	Correct deployment or unstated external behavior
`ReproducibleArtifact`	Independent builds reproduce registered bytes	Behavioral correctness of those bytes
`TrustedTranscription`	A typed external round trip under an explicit TCB	Proof or refinement
`HumanReview`	A named reviewer assessed a bounded scope	Automatic strengthening of other evidence

The essential introduction rules make forbidden coercions syntactically visible. Representative rules are

$$\frac{\Gamma \vdash I \text{ registered} \quad \Gamma; E \vdash O \text{ completes}(I,P)}{\Gamma; E \vdash \mathsf{Sampled}(I,P,O) \,:\, \mathsf{Tested}} \ \textsc{Sample} \tag{4}$$

$$\frac{\Gamma; E \vdash \mathsf{check}(\varphi,\beta) = \mathsf{accept}}{\Gamma; E \vdash \mathsf{BMC}(\varphi,\beta,O) \,:\, \mathsf{BoundedChecked}(\beta)} \ \textsc{Bmc} \tag{5}$$

$$\frac{\Theta \vdash p \,:\, \varphi}{\Gamma; E \vdash \mathsf{Theorem}(\varphi,\Theta,p) \,:\, \langle \mathsf{Proved}, \mathsf{ModelOnly}, \Theta\rangle} \ \textsc{Theorem}. \tag{6}$$

There is no rule from (4) or (5) to Proved, and (6) does not conclude ArtifactBound. This absence is semantically load-bearing. Tests and bounded checks are never treated as proofs; a theorem about a model is not a claim about shipping bytes without an independently registered correspondence. Opaque backend facts may be retained for audit and invalidation, but cannot add a constructor, derivation edge, status role, or policy rule.

5.4 Sampling as intent, plan, facts, and consumers

EXP-0006–0008 refine sampled evidence into the tuple

$$S_b = \langle I, P_b, O_b, R\rangle, \tag{7}$$

where $I = \langle g, X, \sigma, N, \rho, \kappa\rangle$ is common intent (generator closure, target inventory, seed, requested successful-case budget, persistence posture, and empirical ceiling), P_b is a closed plan for backend b, O_b is an authority-indexed fact map, and R is an admission rule. Plans retain controls that do not admit a sound common projection: Hypothesis phases and database mode, fast-check random type and examples, or proptest RNG algorithm and rejection and shrink limits.

For a quantity q, the fact domain is

$$\mathrm{Fact}_b(q) \,::=\, \mathsf{Observed}(v,\alpha) \mid \mathsf{Derived}(v,r,\Delta) \mid \mathsf{Unavailable}(\rho), \tag{8}$$

where α identifies the observation authority, r the derivation rule, and Δ its exact dependencies. In particular, $\mathsf{Unavailable}(\rho) \neq \mathsf{Observed}(0,\alpha)$. An admission rule declares a consumer set $\mathrm{use}(R) \subseteq \mathrm{dom}(O_b)$. It may conclude a sampled judgment only when every required fact is available with an accepted authority:

$$\frac{q \in \mathrm{use}(R) \Rightarrow O_b(q) \in \{\mathsf{Observed}(-,-), \mathsf{Derived}(-,-,-)\} \qquad O_b(\mathsf{completed}) \geq N}{\Gamma; E \vdash S_b \,:\, \mathsf{Tested}[\kappa]}. \ \textsc{Sample-Admit} \tag{9}$$

This gives a consequence-local definition of missing telemetry. Let $\mathrm{deps}_D(r)$ be the transitive dependency cone of a root r in trace D. Then

$$\text{Alert}(q, r) \mathit{iff}\, q \in \text{deps}_D(r) \ \wedge\ O_b(q) = \mathsf{Unavailable}(\rho). \tag{10}$$

Equation (10) is a candidate operational semantics, not a demonstrated claim about human attention. EXP-0008 and EXP-0009 validate it only over their finite attack corpora.

5.5 Canonical encoding and exact joins

Canonical representation is part of semantics because identities, references, and caches depend on bytes. The draft requires strict UTF-8 JSON, recursively sorted object keys, no duplicate keys, explicit required-nullable fields, canonical ordering and duplicate rejection for sets, float rejection, normalized relative paths, domain-separated hashes, and byte-identical round trips. Unknown required schemas and enum values fail closed.

For schema domain d, identity is defined over canonical bytes:

$$\text{id}_d(x) = \text{SHA256}\big(d \parallel \texttt{0x00} \parallel \text{canon}(x)\big). \tag{11}$$

Domain separation prevents a byte string valid in one identity namespace from being substituted in another. An artifact is

$$a = \langle n, h, z \rangle, \qquad a \equiv a' \iff n = n' \wedge h = h' \wedge z = z', \tag{12}$$

where n is the logical name, h the SHA-256 digest, and z the byte size. Digest equality alone is therefore not artifact equality in this calculus. Specification 0001's artifact-bound route additionally checks theorem-statement encoding and digest, exact outer binding form and literal claim/artifact arguments, the artifact-correspondence record, and equality with exactly one provenance input (Proofbound project 2026g). Its characteristic rule is

$$\frac{\Theta \vdash p : \varphi \qquad \Gamma; E \vdash r : \varphi \sim_r a \qquad a \equiv a_{\text{prov}}}{\Gamma; E \vdash \langle p, r, a \rangle : \langle \mathsf{Proved}, \mathsf{ArtifactBound}, \Theta \rangle}. \quad \textsc{Artifact-Bind} \tag{13}$$

Cache validity is the same species of judgment. If $\text{cl}_\Gamma(x)$ is the typed transitive dependency closure, reuse requires equality of the canonical closure, execution configuration, tool and adapter identities, and prior receipt where applicable:

$$\text{Reuse}(x, x') \Rightarrow \text{canon}(\text{cl}_\Gamma(x)) = \text{canon}(\text{cl}_\Gamma(x')). \tag{14}$$

Policy validation analogously joins the selected policy, required facets, cited evidence, assumptions, and blockers.

These joins are load-bearing because self-consistent rehashing is not enough. An attacker can alter a target, artifact, assumption, or policy and recompute local hashes; the independent checker must still reject the changed relation. EXP-0005's adversarial cases include subject, artifact, family, cache, policy, closure, TCB, and status substitutions precisely for this reason.

5.6 Derivation traces

An EXP-0009 derivation program is a rooted directed acyclic graph

$$D = \langle V_F \uplus V_J, E_D, r \rangle, \tag{15}$$

where V_F are canonical, authority-bearing fact nodes, V_J are rule applications, E_D contains exact identity references, and $r \in V_F \uplus V_J$ is the declared conclusion. The serialization is a topological list; its identity is obtained from (11) in the trace domain. Rules cover evidence validation, sampled and bounded testing, theorem checking, mutation witnesses, transcription linkage, artifact binding, assumption derivation, and policy admission.

Replay is a partial function

$$K_\Sigma : \text{Bytes} \rightharpoonup \langle r, \text{Judgment}, \text{Diagnostics} \rangle, \tag{16}$$

parameterized only by closed schema and rule table Σ. It decodes canonically, rejects duplicate identities and cycles, reconstructs each rule output from its predecessors, validates authorities, and requires the recomputed root to equal r. No adapter-authored status is trusted.

The desired backend-erasure property can now be stated. Let α_b be the typed adapter from backend b into the common IR and let erase remove backend plan details that carry no derivation authority. For any two accepted inputs in the domain of a common rule,

$$\text{erase}(\alpha_{b_1}(x_1)) = \text{erase}(\alpha_{b_2}(x_2)) \Longrightarrow K_\Sigma(\alpha_{b_1}(x_1)) = K_\Sigma(\alpha_{b_2}(x_2)). \tag{17}$$

Equation (17) is a design obligation, not a proved metatheorem. The registered EXP-0009 corpus supplies finite evidence for it: its common rule table contains no concrete backend name, and the Rust and Python replayers agree on every generated conclusion and rejection. The trace is consequently a portable check object and explanation, not a self-authenticating certificate.

5.7 Invalidation semantics

Let Δ be the set of identities whose canonical values or availability states have changed. Invalidation is the upward closure of Δ in the exact dependency graph:

$$\text{Inv}_D(\Delta) = \mu X.\ \Delta \cup \{v \mid \exists u \in X.\, (u, v) \in E_D\}. \tag{18}$$

A cached judgment rooted at r is reusable only if $r \notin \text{Inv}_D(\Delta)$ and the closure equality (14) holds. The principal safety obligation is *no false retention*:

$$\text{deps}_D(r) \cap \Delta \neq \varnothing \Longrightarrow r \in \text{Inv}_D(\Delta). \tag{19}$$

Precision is distinct: invalidating the entire repository satisfies (19) but destroys useful reuse. The intended optimization criterion is therefore to minimize $|\text{Inv}_D(\Delta)|$ subject to (19) and exact typed dependencies. This semantics is proposed. EXP-0005 contains bounded cache-closure and substitution attacks, but the dedicated invalidation experiment required to validate (19) over code, tools, permissions, configuration, assumptions, policy, and external-contract changes has not yet been executed.

6. Experimental Method

6.1 Preregistration and falsifiers

Each experiment commits questions, pass criteria, falsifiers, corpus scope, and procedure before execution. Journals are append-only; corrections are later entries. Machine-readable results are immutable files. The experiment index in `docs/experiments/README.md` distinguishes `planned`, `running`, `concluded`, and `abandoned`. Some README headers preserve preregistration-era state, so this paper uses the append-only journal, immutable result, and current index to identify execution status rather than rewriting the registration after observing results.

This method limits post-hoc success criteria. It does not eliminate researcher degrees of freedom: the same project designed the hypotheses, prototypes, and corpora. Section 9 treats that as a validity threat.

6.2 Positive, adversarial, and holdout corpora

Positive cases test representability and expected derivation. Adversarial cases modify one semantic relationship or encoding and preregister the rejection class. Attacks include omission, duplicate identity, alias or substitution, changed assumption, subject/artifact mismatch, inventory skew, cache incompleteness, unknown rules, cycles, noncanonical encoding, stronger reported roots, and legacy schema reinterpretation. This follows differential-testing practice (McKeeman 1998) but compares semantic decoders and derivations rather than compiler output alone.

EXP-0007 is a third-ecosystem holdout. The contract first demonstrated on Hypothesis and fast-check was frozen before testing proptest 1.11.0. The study stopped once preregistered falsifiers for its questions had fired instead of expanding the common contract to force a pass.

EXP-0009 moves beyond hand-selected cases. Six route templates drive a deterministic generator (`proofbound-exp-0009-generator/1`, seed 9009). It emits 500 valid programs and 500 adversarial programs, each with one mutation. The sixteen attack classes occur 31 or 32 times each, which is the exact partition of 500 cases.

6.3 Independent implementations

The Rust prototype is the repository's `proofbound-ir-prototype` crate. The independent `assurance_ir_checker.py` lives under `python/proofbound/`; it has its own decoder and rule tables and does not import producer types or generated bindings. Independence is architectural, not statistical: two implementations can share a mistaken specification or design premise. Agreement therefore detects divergences and ambiguous encodings in the registered domain but is not proof of correctness.

6.4 Evidence snapshot and artifact ledger

All numerical and technical claims in this paper were checked against the repository snapshot below. Uncommitted working-tree changes are excluded.

```
e54f428601f53e065262f74b97bba3d7e2b34150
```

Table 3 gives the exact ledger IDs and repository-relative paths for the principal result artifacts. Each experiment's `ARTIFACTS.md` records the full 64-digit raw SHA-256 identity.

Ledger ID	Exact checked-in result path
`EXP-0005-A010`	`docs/experiments/0005-assurance-ir-extraction/results/2026-09-02-adversarial-evidence-algebra.json`
`EXP-0005-A023`	`docs/experiments/0005-assurance-ir-extraction/results/2026-09-02-q1-completion-capture-audit.json`
`EXP-0005-A026`	`docs/experiments/0005-assurance-ir-extraction/results/2026-09-02-portable-family-projection.json`
`EXP-0006-X003`	`docs/experiments/0006-explicit-sampling-contract/results/2026-09-02-adapter-owned-driver.json`
`EXP-0007-X001`	`docs/experiments/0007-rust-sampling-holdout/results/2026-09-02-proptest-holdout.json`
`EXP-0008-X001`	`docs/experiments/0008-layered-sampling-model/results/2026-09-02-layered-sampling-model.json`
`EXP-0009-X001`	`docs/experiments/0009-generated-evidence-algebra/results/2026-09-03-generated-evidence-algebra.json`

6.5 Interpretation discipline

Four levels of statement are used below.

- **Implemented:** present in the production verifier or a named research prototype at the pinned commit.
- **Bounded experimental evidence:** an observed result over the exact registered corpus and implementation.
- **Hypothesis:** a falsifiable programme claim not yet established.
- **Proposal:** a future design or roadmap step without current empirical support.

Passing tests validate the implementation against cases; they are never reported as proofs. A bounded model-check result is always reported with its bounds and never as an unbounded theorem. Likewise, independent implementation agreement is not independence of underlying assumptions.

7. Results

7.1 Algebraic separation and representation loss (EXP-0005)

EXP-0005 began with 20 positive projection cases spanning Python, TypeScript, Rust, Kani, Lean, mutation, distribution, transcription, refinement semantics, and proof-free release/compiler cases. Independent Rust and Python implementations agreed on all 20 positives, rejected all 20 registered adversarial cases with the expected codes, and agreed on 15 canonical domain-hash vectors (three JSON values across five domains). There were zero producer/checker disagreements and zero concrete backend-name hits in the generic kernel over that bounded corpus (Proofbound project 2026a). This supports RQ2 and the backend-neutral portion of RQ1; it does not establish semantic sufficiency.

The losslessness audit is the controlling negative result. Revision 1 found only one of 16 rows forward-complete and none reverse-complete. After reverse projection and representation hardening, revision 3 reached 12 complete rows, but full Python, TypeScript, and Rust release captures reopened typed portable-family coverage. The current checked-in revision 4 records 11 of 16 rows forward-and-reverse complete and five partial:

- publication decision and complete blocking derivation (`Q1-FIELD-006`);
- registration-to-observation artifact identities (`Q1-FIELD-009`);
- complete typed portable evidence-family details (`Q1-FIELD-012`);
- transitive execution/cache dependencies and retained reuse eligibility (`Q1-FIELD-014`); and
- complete admission explanation and derivation trace (`Q1-FIELD-016`).

Consequently Q1 is failed at the current decision point, Assurance IR `/1` is not frozen, and the Go holdout has not begun. The ordering matters: successful adversarial parity on a partial representation does not make the representation lossless.

A later closed portable-family projection covered all 45 captured records with identical Rust/Python output. Its constructor counts were: 12 examples, seven mutation witnesses, seven universal source proofs, six human reviews, three sampled properties, two artifact correspondences, two independent observations, two reproducible artifacts, two static-consistency records, one bounded model check, and one trusted transcription. Of the three property records, only one had explicit sampling semantics; the TypeScript and Rust records remained `LegacyBackendSampling`. A self-consistently rehashed attempt to upgrade legacy sampling was rejected.

Finite-model proposition 1 (Non-collapse over the EXP-0005 domain)**.** *No semantics-preserving projection from the registered heterogeneous evidence records to a generic success constructor was exhibited. Every registered family substitution and illicit status strengthening was rejected by both checkers, while the bidirectional audit identified information not retained by a generic* `passed` *envelope.*

This proposition is quantified only over the EXP-0005 corpus. It supports the closed sum (3); it does not establish completeness of that sum.

7.2 Authority-preserving observation (EXP-0006)

EXP-0006 tested Hypothesis 6.112.0 and fast-check 4.3.0. Ordinary runner output failed Q1 and Q2; Vitest setup instrumentation failed closed. An adapter-owned generator/predicate driver then controlled seed, budget, persistence and shrinking policy, framework execution, counters, and exclusive report creation.

Both positive runs completed exactly 100 cases. Hypothesis used seed 4025493768; fast-check used seed 424242. Both observations recorded 100 attempted, 100 completed, zero skipped, and zero shrinks. Deliberately false properties produced typed counterexamples and exited 1: the Hypothesis run made two predicate invocations and the fast-check run one, with fast-check recording one shrink. Rust and Python validators rejected all ten preregistered attacks with their exact classes and introduced no framework-name branch in common validation (Proofbound project 2026b).

Finite-model proposition 2 (Observation-boundary sufficiency for two backends)**.** *For Hypothesis 6.112.0 and fast-check 4.3.0, the adapter-owned driver retained the registered sampling intent and authoritative observations and rejected all ten registered mutations; ordinary runner reports did not determine the same record.*

Consequently, tool name and seed do not determine the observed sampling term. Generator and target identity, requested and completed budget, persistence, shrinking policy, effective execution controls, observations, and authoring authority are semantically relevant. Production use would additionally require outer command, environment, driver, runtime, and framework provenance.

7.3 Counterexample to a flat sampling contract (EXP-0007)

EXP-0007 froze the EXP-0006 contract and evaluated proptest 1.11.0. With seed 424242, `chacha` RNG, disabled persistence, 100 successful cases, shrink limit 10,000, and local/global rejection limits of 1,000, the positive run made 100 predicate invocations. A counterexample run made 23 and minimized to the recorded all-zero/false value. Changing only the RNG algorithm to `xorshift` kept the framework version and seed fixed but produced a distinct execution with 25 counterexample predicate invocations. The EXP-0006 contract identity would not have changed because RNG algorithm was absent (Proofbound project 2026c).

The stable typed proptest API exposed cases, persistence, rejection limits, shrink limit, RNG algorithm and seed, plus pass/failure and the minimal failing value. It did not expose authoritative successful-case, local/global rejection, or accepted-shrink counters in the required form. The study rejected parsing human `Display` output, accessing private fields, and treating every predicate invocation as a fresh attempt. After attack `EXP-0007-A002` triggered the preregistered falsifiers, the remaining eleven attacks were not executed because they could not restore the unchanged contract's generality.

Finite-model proposition 3 (Flat-contract falsification)**.** *The EXP-0006 identity function is not injective with respect to relevant proptest executions: holding framework version and seed at 1.11.0 and 424242 while changing only the RNG algorithm from `chacha` to `xorshift` changed the counterexample execution from 23 to 25 predicate invocations without changing the frozen contract identity.*

Thus common semantic intent does not entail identical backend execution records. Backend-specific controls must remain typed, and fact terms must state which observations a backend can authoritatively supply.

7.4 Layered facts and consequence locality (EXP-0008)

EXP-0008 projected frozen Hypothesis, fast-check, and proptest records into the intent/plan/authority model. The three corpus files were 2,203, 2,199, and 2,444 bytes respectively for Hypothesis, fast-check, and proptest, with exact hashes recorded in the result. The common intent retained ceiling, generator, persistence, seed, successful budget, and targets. Closed backend plans retained the divergent execution controls. Common validation and admission contained zero framework-name branches.

Hypothesis and fast-check reported attempted, completed, skipped, and shrink facts as observed. For proptest, attempted, skipped, and shrink facts were unavailable; completion was derived from the runner-success contract, intent budget, and typed pass result. The admission rule consumed only `completed` and set the ceiling to `empirical-sample`.

Rust and Python agreed on all twelve registered attacks. Notably, `EXP-0008-A007` made completed budget unavailable and was rejected as `sampling-admission-blocked`; `EXP-0008-A009` removed unused shrink telemetry and returned `no-admission-consequence`. Legacy Rust and EXP-0006 records were rejected as layered schema rather than reinterpreted (Proofbound project 2026d).

Finite-model proposition 4 (Consequence locality over twelve attacks)**.** *For the three registered framework records and twelve mutations, both checkers blocked admission exactly when an unavailable fact belonged to the admission dependency cone. Removing unavailable shrink telemetry outside that cone did not alter the conclusion.*

This validates (10) for the registered cases. It does not establish that the corresponding diagnostic policy reduces notification fatigue in human use.

7.5 Closed-trace replay and backend erasure (EXP-0009)

EXP-0009 used six templates: sampled property, bounded check, theorem, mutation witness, trusted transcription, and artifact binding. The deterministic generator created 500 valid and 500 single-mutation adversarial programs under seed 9009. The corpus covered eleven rules and all sixteen attack classes; attacks occurred 31 or 32 times each. Independent Rust and Python implementations accepted every valid program with identical conclusions and trace identities and agreed on the expected rejection or no-consequence result for every adversarial program. The reported disagreement count and backend-named common-rule count were both zero (Proofbound project 2026e).

The attacks exercised more than evidence-family substitution. They removed or replaced exact dependencies, substituted an artifact binding, removed an open assumption, introduced a cycle, duplicated an identity, used an unknown backend-named rule, omitted the source of a derived fact, changed the declared root, introduced noncanonical encoding, attempted to strengthen transcription, and reinterpreted an older schema. `EXP-0009-A011` made a consumed fact unavailable and blocked its exact derivation; `EXP-0009-A012` removed unused duration telemetry and preserved the conclusion without an alert.

Finite-model proposition 5 (Differential replay over the generated corpus)**.** *For all 1,000 generated programs in EXP-0009, the Rust and Python replayers agreed on acceptance, conclusion, trace identity, or registered rejection class. The common rule table contained zero concrete backend-name branches.*

Exact references, canonical order, authority checks, and root validation are therefore sufficient for deterministic replay over this registered slice. The finite result does not prove the algebra sound, complete, or equivalent to every production route.

7.6 Cross-experiment synthesis

Finding	Strongest checked-in support	Epistemic status
Evidence families must not flatten to `passed`	EXP-0005 family/status attacks; EXP-0009 coercion attacks	Bounded experimental evidence
Sampling needs more than tool name and seed	EXP-0006 observation failures; EXP-0007 RNG falsifier	Direct negative and bounded positive evidence
Facts need authority, availability, and explicit consumers	EXP-0008 twelve attacks; EXP-0009 A010–A012	Bounded experimental evidence
Closed traces can be backend-independent	EXP-0009 six routes, eleven rules, 500/500 programs	Bounded experimental evidence
Unused unavailable telemetry should not alert	EXP-0008 A009; EXP-0009 A012	Implemented research semantics, not a user outcome
Legacy records must remain visibly weaker	EXP-0005 1 explicit/2 legacy sampling records; EXP-0006 Q4; EXP-0008 A011–A012	Bounded migration evidence
Exact joins are essential	EXP-0005 subject/artifact/cache/policy attacks; Specification 0001 verifier contract	Implemented production checks plus bounded attacks

8. Discussion

8.1 Why assurance requires language semantics

JSON or TOML can serialize the terms above; serialization is not the contribution. The language problem is the definition of well-formed judgments, admissible coercions, effect and authority boundaries, module composition, invalidation, and diagnostic consequence. A frontend should reject, before execution, a sampled term supplied where policy requires a universal theorem, or a supposedly hermetic action whose effect row includes network authority. These are typing failures, not post-hoc dashboard warnings.

The semantic kernel is therefore prior to surface syntax. A native frontend is acceptable only as a conservative elaboration into Assurance IR. For surface language L, the minimum criterion is

$$\Gamma \vdash_L p : j \Longrightarrow K_\Sigma(\mathrm{canon}([\![p]\!]_L)) = j, \tag{20}$$

with no additional trusted status rule in the compiler. A polished syntax that does not meet (20) merely conceals semantic omissions behind source compatibility.

8.2 The Tower of Babel and the kernel boundary

Cross-ecosystem assurance naturally accumulates dialects: pytest nodes, Vitest selectors, Hypothesis phases, fast-check random types, proptest RNG algorithms, Kani harness metadata, Lean theorem environments, Aeneas translation reports, wheel rules, npm package rules, and tool-specific availability conditions. Putting each directly into the kernel yields a union of backend schemas rather than a language-neutral semantics. H1's explicit falsifier is such proliferation.

Let Σ_b be the backend-specific schema and Σ_K the common kernel signature. An unconstrained integration approaches the coproduct $\coprod_b \Sigma_b$: every framework construct becomes a kernel construct, and status derivation grows by backend case analysis. This is the Tower-of-Babel failure mode named by H1.

Containment requires two constraints.

Small semantic kernel. Common meaning is limited to typed claims, subjects, identities, evidence constructors, authorities, derivation rules, and policy decisions. The kernel rejects unknown required semantics and never branches on concrete backend names. EXP-0005, EXP-0008, and EXP-0009 report zero such branches in their tested common validators.

Typed backend boundaries. Diversity is retained in closed backend plan/detail variants and provenance. A backend adapter may know how Hypothesis represents phases or how Kani enumerates harnesses; it cannot decide that the observation is a proof. A new backend either maps losslessly to an existing evidence constructor with a typed plan and capability description, or motivates an explicit versioned semantic change. Opaque fields may affect audit and invalidation but cannot acquire derivation authority.

Equivalently, growth is permitted in the family $\{\alpha_b : \Sigma_b \rightharpoonup \Sigma_K\}_b$, not in the codomain Σ_K without an explicit versioned semantic extension. This does not make adapters correct or prevent backend schemas from multiplying. It confines that complexity outside the smallest status-checking boundary and renders its effects amenable to differential and adversarial validation.

8.3 Exact joins are the semantics

It may appear pedantic to join logical name, digest, and size rather than digest alone, or to preserve plan identity separately from intent. The experiments show why these details are not metadata. A theorem, artifact, source closure, cache entry, and policy can each be valid independently while their composition is invalid. Assurance depends on the relation: *this* theorem, about *this* claim, corresponds to *this* exact input artifact under *this* policy and dependency closure.

Canonical encoding makes local identity reproducible; exact joins make global meaning reproducible. Both are needed. A self-consistent local rehash after a substitution is an attack, not validation.

Migration obeys the same law. Let m_v translate schema version v into the current IR and let $\sqsubseteq$ mean "contains no stronger assurance meaning than." A migration is admissible only if

$$m_v(e) \sqsubseteq e \oplus e_{\text{new}}, \tag{21}$$

where e_{new} is genuinely new, independently identified evidence. For a legacy property record with no explicit sampling plan, $e_{\text{new}} = \varnothing$ forbids $m_v(e) = \mathsf{Sampled}(I, P, O)$. The only honest image is a visibly weaker legacy constructor. EXP-0005's self-consistently rehashed upgrade attack and the legacy-schema attacks in EXP-0008 and EXP-0009 exercise this non-escalation condition.

8.4 Practical adoption: bridge before destination

The practical path begins with existing systems. Proofbound can register claims over Python, TypeScript, and Rust; observe pytest, Hypothesis, Vitest, fast-check, static analyzers, and mutation witnesses; integrate Kani bounded checks; retain Aeneas/Charon translation and refinement boundaries; ingest Lean theorem evidence; and bind reproducible distributions and artifacts. These integrations are heterogeneous by design. They create the empirical corpus from which stable language semantics can be extracted.

Mixed-language systems are not a transitional embarrassment; they are the principal semantic case. A future DSL can author assurance programmes for foreign code. A future native component can compile into the same IR and coexist

with Python or TypeScript callers. The native route is justified only if it measurably strengthens correspondence, static effect control, usability, or notification quality without inflating the trusted base or proof burden. Until then, the bridge is the principal system and research instrument.

8.5 Publication, uncertainty, and human judgment

The proposed model does not abolish uncertainty. Stakeholder intent, hardware, external services, operators, credentials, and organizational processes remain outside many formal models. The language vision makes assumptions, exclusions, owners, expiry, and consequences first-class so they can participate in invalidation and publication decisions.

Human review is similarly typed as evidence about a declared scope. It may acknowledge an exception or evaluate an assumption; it does not upgrade sampled evidence into proof. A useful notification should name the affected claim, dependency path, failed rule, and requested decision. Whether this presentation actually improves decision quality is open research, not a conclusion from the current corpora.

9. Threats to Validity and Limitations

9.1 Construct validity

The experiments operationalize semantic sufficiency through registered fields, reverse projections, attacks, and status parity. The field inventory may omit a real assurance concept, and the chosen statuses may not capture how organizations reason about risk. The programme's refusal to produce a numeric score prevents one class of false precision but does not demonstrate that its facet structure is complete.

Notification consequences are tested as rule behavior, not as human outcomes. No developer, security engineer, release owner, or auditor study has yet measured missed critical consequences, false escalation, investigation time, or dismissal rates. Claims of reduced notification fatigue would therefore be premature.

9.2 Internal validity

The project authors designed the models, implementations, generators, and most corpora. Independent Rust/Python implementations reduce common-code failure but not common-specification bias. Many EXP-0005 fixtures are projections of existing records without re-executing the original evidence backends. The generated EXP-0009 corpus is deterministic and broad over registered rules, but every adversarial program contains exactly one mutation and is derived from six templates. Multi-fault interactions and unanticipated attacks are underrepresented.

The checked-in worktree includes research prototypes, not a formally verified checker. Unit tests, differential checks, and adversarial corpora provide bounded evidence only. They do not prove implementation soundness.

9.3 External validity

Sampling studies cover Hypothesis 6.112.0, fast-check 4.3.0, and proptest 1.11.0. Other versions and property systems may expose different controls or observation capabilities. The portable corpus contains 45 captured records from three release verticals; the 500/500 derivation corpus spans six routes, not every production constructor. The planned Go holdout has not started because EXP-0005's losslessness gate remains open.

Formal-method integrations are also selective. Lean, Kani, and Aeneas exercise important theorem, bounded, and refinement boundaries, but do not stand for all proof assistants, SMT solvers, abstract interpreters, symbolic executors, or verified compilers. Backend-neutrality is supported only for the tested routes.

9.4 Migration and compatibility limits

The draft `proofbound-assurance-ir/1` schema is explicitly non-normative, partially implemented, and not frozen. No production manifest, receipt, cache, release, or verifier is authorized to emit or accept it. Legacy property receipts preserve their current empirical ceiling and identities but lack explicit sampling meaning. Reclassifying them without new evidence would be a semantic upgrade and is rejected by the prototypes.

Current open gaps include complete publication traces, execution-time artifact identity joins, typed portable-family coverage in the central IR, complete transitive cache dependencies, and admission explanation. The uncommitted prototype work present in the local checkout is outside this paper's evidence snapshot and does not change these reported results.

9.5 Scope limits

Proofbound does not establish that public claim wording matches stakeholder intent, that hardware implements a formal machine model, that external services honor contracts, or that operators deploy the checked artifact and configuration. It can name these as assumptions or correspondence obligations. It cannot remove them by language design.

The system also does not turn deterministic reproduction into behavioral correctness, tool availability into evidence, review into proof, or a successful model theorem into an artifact claim. These are permanent boundaries rather than missing features.

10. Research Roadmap

The roadmap is dependency-ordered, not a delivery schedule (`docs/research/proofbound-language/roadmap.md`).

Gate	Research objective	Exit condition relevant to language work
1. Shared semantics	Finish Assurance IR, evidence algebra, invalidation, and independent checking	Smaller than backend-schema union; every status explicit; zero known false retention
2. Authoring and authority	Compare typed DSLs and prototype effects/capabilities	Equivalent frontends emit identical IR; demonstrated authority defects prevented
3. Product value	Study uncertainty and claim-oriented notifications	Lower irrelevant escalation without more missed critical consequences
4. Native feasibility	Build one small deterministic executable subject and artifact story	Independently checkable assurance stronger than comparable existing-language integration
5. Adoption bridge	Test mixed native/foreign systems and final independent kernel	Honest, usable migration with existing-language Proofbound still first-class

The immediate next experiment is invalidation. It must mutate code, tools, permissions, missing paths, configuration, assumptions, policies, and external contracts; predict affected conclusions from typed dependencies; execute fresh checks; and require zero false retention in the registered corpus. It should also measure invalidation precision so safety does not degenerate into whole-repository reruns.

Only after Gate 1 should an assurance DSL be compared with existing typed configuration approaches such as restricted Pkl or CUE. Equivalent TOML and DSL programmes must compile to byte-identical canonical IR, and diagnostics should make real substitution errors unrepresentable before execution. An effect model then needs to separate static capability claims, operating-system enforcement, and post-execution observations for `Read`, `Write`, `Execute`, environment, network, clock, randomness, secret, and human-judgment authority.

The native experiment, if reached, should be deliberately small: a canonical parser/serializer, policy evaluator, protocol state machine, ledger transition, or capability-token validator. It must compose with an existing Python or TypeScript claim, expose its universal proof obligations separately from examples and sampled properties, and bind the resulting artifact. Failure to outperform a small component in Verus, Lean, Dafny, or another existing verified language is a reason to stop, not to widen the implementation.

The roadmap admits three legitimate outcomes: remain an assurance framework if no compact semantic core survives; ship only a typed assurance DSL if authoring improves but native execution does not; or specify a native language only after mixed-language Gate 5 passes. This staged decision rule is the main defense against premature language design.

11. Conclusion

Software assurance is not the conjunction of successful tool invocations. It is a typed derivation relating a claim to evidence of determinate kind, authority, subject, assumptions, and artifact identity under an explicit publication policy. The Proofbound calculus makes that derivation a first-class, canonical, independently replayable object. Its decisive prohibitions are structural: there is no coercion from testing or bounded checking to proof; no coercion from model theorem to artifact claim; no admission consequence from an unavailable fact outside the root dependency cone; and no migration that strengthens legacy evidence without a new observation.

The checked-in experiments support these prohibitions over precisely delimited domains. EXP-0005 rejects evidence-family and join substitutions while exposing five unresolved losslessness obligations among sixteen. EXP-0006–0008 show why portable sampling requires intent, typed backend plans, authority-indexed facts, availability, and consumers. EXP-0009 obtains agreement between independent Rust and Python replayers over six routes, eleven rules, and 1,000 generated programs, with sixteen attack classes and no backend-named common rule. These finite checks are not proofs of metatheoretic soundness, algebraic completeness, or human benefit; they are unusually explicit evidence for the design obligations that a future formalization must discharge.

The architectural conclusion is nevertheless firm. Backend multiplicity belongs at typed elaboration boundaries, while evidence meaning, canonical identity, derivation, and policy remain in a small kernel. Proofbound should therefore serve first as a semantic bridge over Python, TypeScript, Rust, Lean, Kani, Aeneas, and related systems. A native language is justified only as a conservative surface for this calculus and only where it demonstrably reduces accidental complexity. Anything weaker would not resolve the Tower of Babel; it would merely add another language to it.

References

Bloomfield, Robin, and John Rushby. 2021. “Assurance 2.0: A Manifesto.” *arXiv Preprint arXiv:2004.10474*. https://doi.org/10.48550/arXiv.2004.10474.

Cutler, Joseph, Craig Disselkoen, Aaron Eline, Shaobo He, Kyle Headley, Mike Hicks, Kesha Hietala, et al. 2024. “Cedar: A New Language for Expressive, Fast, Safe, and Analyzable Authorization.” *arXiv Preprint arXiv:2403.04651*. https://doi.org/10.48550/arXiv.2403.04651.

Ho, Son, and Jonathan Protzenko. 2022. “Aeneas: Rust Verification by Functional Translation.” *Proceedings of the ACM on Programming Languages* 6 (ICFP). https://doi.org/10.1145/3547647.

Kelly, Tim, and Rob Weaver. 2004. “The Goal Structuring Notation – a Safety Argument Notation.” In *Proceedings of the DSN 2004 Workshop on Assurance Cases*. https://www-users.york.ac.uk/~tpk1/dsn2004.pdf.

Lattuada, Andrea, Travis Hance, Jay Bosamiya, Matthias Brun, Chanhee Cho, Hayley LeBlanc, Pranav Srinivasan, et al. 2024. “Verus: A Practical Foundation for Systems Verification.” In *Proceedings of the 30th ACM Symposium on Operating Systems Principles*, 438–54. https://doi.org/10.1145/3694715.3695952.

McConnell, Ross M., Kurt Mehlhorn, Stefan Näher, and Pascal Schweitzer. 2011. “Certifying Algorithms.” *Computer Science Review* 5 (2): 119–61. https://doi.org/10.1016/j.cosrev.2010.09.009.

McKeeman, William M. 1998. “Differential Testing for Software.” *Digital Technical Journal* 10 (1): 100–107.

Mokhov, Andrey, Neil Mitchell, and Simon Peyton Jones. 2018. “Build Systems à La Carte.” *Proceedings of the ACM on Programming Languages* 2 (ICFP). https://doi.org/10.1145/3236774.

Moura, Leonardo de, and Sebastian Ullrich. 2021. “The Lean 4 Theorem Prover and Programming Language.” In *Automated Deduction – CADE 28*, 12699:625–35. Lecture Notes in Computer Science. Springer. https://doi.org/10.1007/978-3-030-79876-5_37.

Necula, George C. 1997. “Proof-Carrying Code.” In *Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, 106–19. https://doi.org/10.1145/263699.263712.

Proofbound project. 2026a. “Experiment 0005: Assurance IR Extraction.” Preregistration, append-only journal, corpora, and results. https://github.com/bordumb/proof-bound.

———. 2026b. “Experiment 0006: Explicit Sampled-Property Contract.” Preregistered experiment and immutable result at commit 6a35518. https://github.com/bordumb/proof-bound.

———. 2026c. “Experiment 0007: Rust Sampled-Property Holdout.” Preregistered experiment and immutable result at commit dcdffb7. https://github.com/bordumb/proof-bound.

———. 2026d. “Experiment 0008: Layered Sampling Model.” Preregistered experiment and immutable result at commit d6da0b8. https://github.com/bordumb/proof-bound.

———. 2026e. “Experiment 0009: Generated Evidence Algebra.” Preregistered experiment and immutable result at commit e54f428. https://github.com/bordumb/proof-bound.

———. 2026f. “Proofbound Language Research Programme.” Research programme at commit e54f428. https://github.com/bordumb/proof-bound.

———. 2026g. “Specification 0001: Initial Specification.” Repository specification at commit e54f428. https://github.com/bordumb/proof-bound.

Rondon, Patrick M., Ming Kawaguchi, and Ranjit Jhala. 2008. “Liquid Types.” In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation*, 159–69. https://doi.org/10.1145/1375581.1375602.

Torres-Arias, Santiago, Hammad Afzali, Trishank Karthik Kuppusamy, Reza Curtmola, and Justin Cappos. 2019. “In-Toto: Providing Farm-to-Table Guarantees for Bits and Bytes.” In *28th USENIX Security Symposium*, 1393–1410. USENIX Association. https://www.usenix.org/conference/usenixsecurity19/presentation/torres-arias.